

Cours IFAL Serveur Web

Thierry Boulanger

12 avri 2026



Administration et Sécurisation des Serveurs Web

Support de Cours - Avril 2026

Introduction

Un serveur web n'est pas seulement un logiciel qui envoie des fichiers ; c'est le point de contact critique entre une infrastructure privée et l'Internet public. Administrer un serveur, c'est garantir que ce point de contact reste performant, disponible et, par-dessus tout, **hermétique aux attaques**.

Objectifs Opérationnels

Le but est de maîtriser le cycle de vie complet d'un service web :

1. **Déploiement & Configuration** : Installer et optimiser les environnements (Apache, IIS, Node.js).
2. **Gestion de Contenu** : Administrer des CMS et des bases de données de manière structurée.
3. **Sécurisation** : Appliquer les protocoles HTTPS, gérer l'authentification et les accès.
4. **Résilience** : Planifier les sauvegardes et assurer la migration sans perte de service.

I. Architecture des Serveurs Modernes

1.0 Le développement web

Le développement web repose sur une synergie entre les langages interprétés par le navigateur de l'utilisateur et ceux exécutés sur la machine distante. Voici une analyse détaillée de ces piliers technologiques :

Le Front-End : HTML et CSS, le binôme indissociable

Le **HTML** (*HyperText Markup Language*) constitue la structure fondamentale de toute page web. Ce n'est pas un langage de programmation, mais un langage de balisage : il sert à sémantiser le contenu. Chaque balise (comme `<h1>` pour un titre ou `<p>` pour un paragraphe) définit la nature de l'information. En 2026, l'**HTML5** reste la norme, offrant des fonctionnalités natives pour l'audio, la vidéo et le stockage local, réduisant ainsi la dépendance aux plugins tiers.

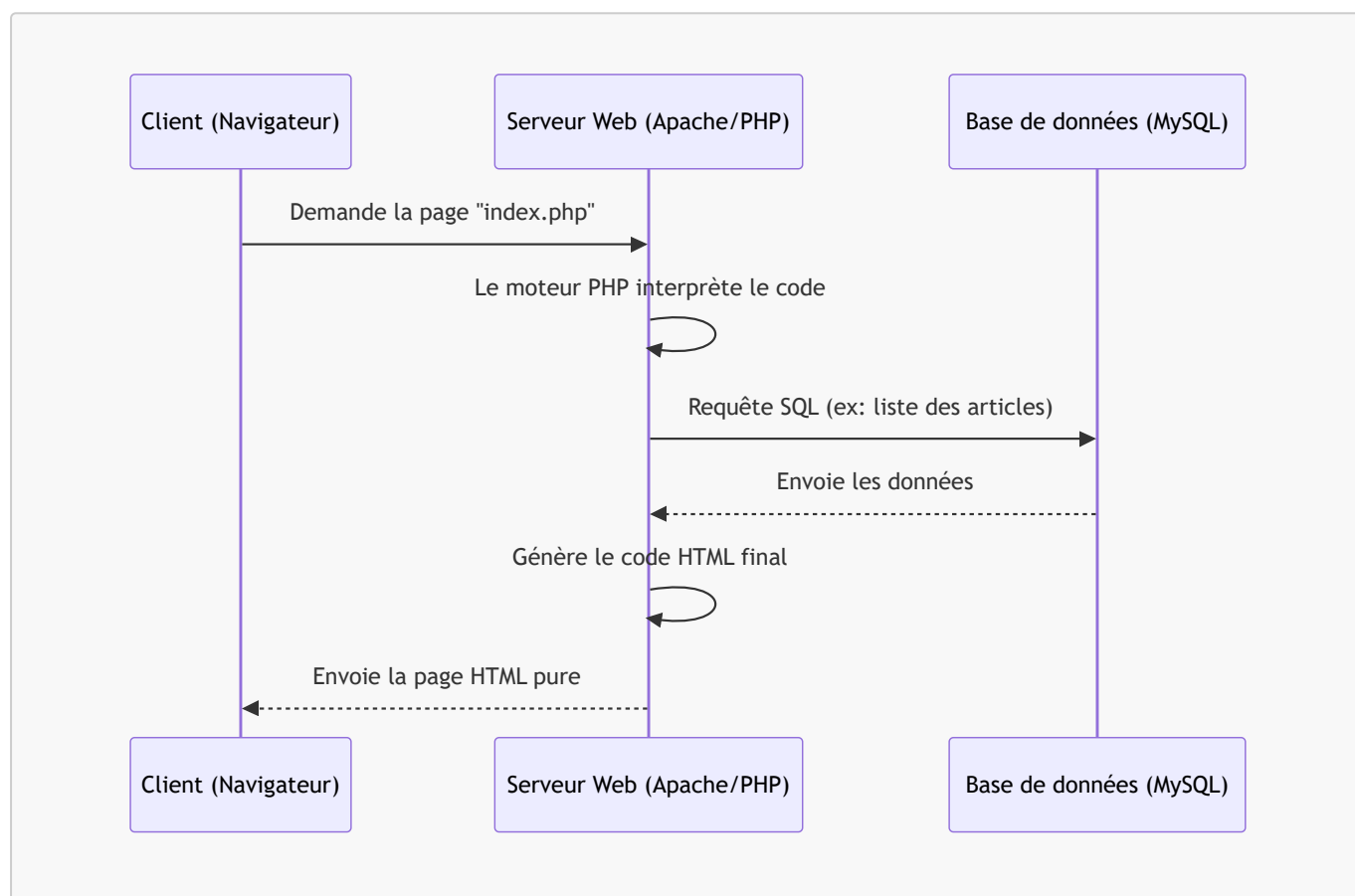
Le **CSS** (*Cascading Style Sheets*), quant à lui, est le langage de la mise en forme. Sans lui, le web ne serait qu'une suite de textes bruts. Il permet de contrôler la mise en page (grâce à des outils modernes comme **Flexbox** et **Grid**), les couleurs, les polices et les animations. La séparation stricte entre le fond (HTML) et la forme (CSS) est une règle d'or pour assurer la maintenabilité d'un projet et son accessibilité sur tous les terminaux (ordinateurs, tablettes, smartphones).

Les langages serveurs : L'intelligence invisible

Contrairement au Front-End qui s'exécute chez le client, les **langages serveurs** (Back-End) traitent les données avant même que la page ne soit envoyée au navigateur. Ils gèrent la logique métier : authentification des utilisateurs, paiements sécurisés, ou récupération d'articles dans une base de données. Outre le PHP, on retrouve aujourd'hui des environnements puissants comme **Node.js** (JavaScript côté serveur), **Python** (avec Django) ou **Ruby**, chacun apportant ses propres avantages en termes de vitesse de développement ou de performance.

Le cas spécifique du PHP : Le moteur historique

Le **PHP** (*Hypertext Preprocessor*) demeure l'un des langages les plus emblématiques du web, propulsant près de **80%** des sites internet mondiaux, notamment grâce à des géants comme WordPress ou Facebook. Sa particularité réside dans sa capacité à s'insérer directement au milieu du code HTML pour le rendre dynamique.

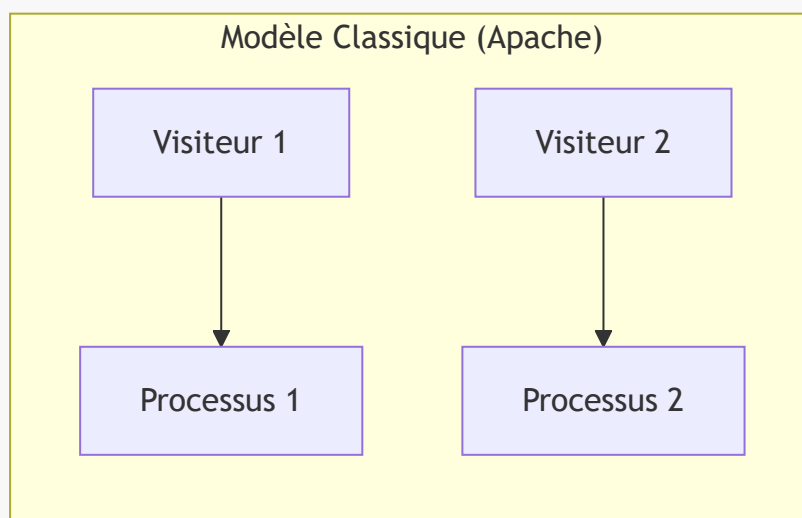
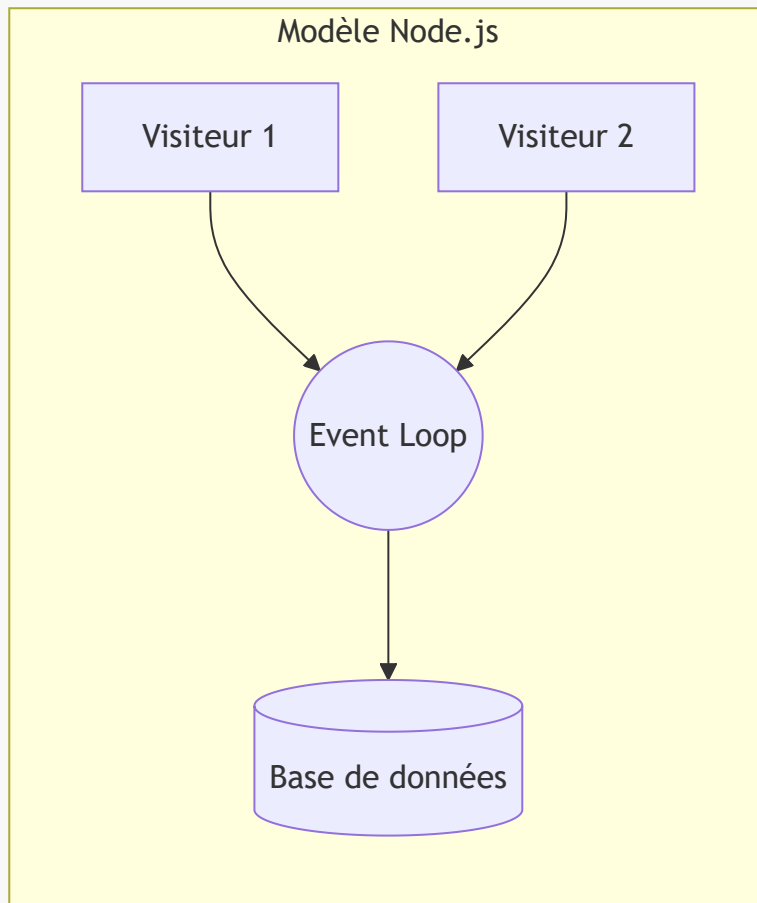


Aujourd'hui, avec la version **PHP 8.x**, le langage a considérablement gagné en performance et en sécurité. Il ne se limite plus à de petits scripts : grâce à des frameworks robustes comme **Symfony** ou **Laravel**, PHP permet de construire des architectures professionnelles complexes, respectant les plus hauts standards de la sécurité informatique moderne (programmation orientée objet, typage strict et protection native contre les injections).

1.1 Le duel des approches : Classique vs Événementiel

Historiquement, le web reposait sur des serveurs "multi-threadés" (un processus par visiteur). Aujourd'hui, l'architecture a évolué pour répondre à des besoins de haute performance.

- **Serveurs Classiques (Apache / IIS) :** * **Apache :** Le standard open-source. Très modulaire (utilisation de `.htaccess` pour configurer dossier par dossier).
 - **IIS (Internet Information Services) :** La solution Microsoft, intégrée à l'écosystème Windows Server et Active Directory.
- **Serveurs Modernes (Node.js) :** * **Définition :** Node.js n'est pas un serveur web en soi, mais un **environnement d'exécution** JavaScript.
 - **Fonctionnement :** Il utilise une boucle d'événements unique (*Event Loop*). Au lieu d'attendre qu'une base de données réponde, il passe à la requête suivante. C'est le modèle **Non-bloquant (Asynchrone)**.



II. L'Écosystème Applicatif : CMS et Frameworks

2.1 Les CMS (Content Management Systems)

Un **CMS** est un logiciel installé sur le serveur qui permet de séparer le contenu (textes, images) de la structure technique.

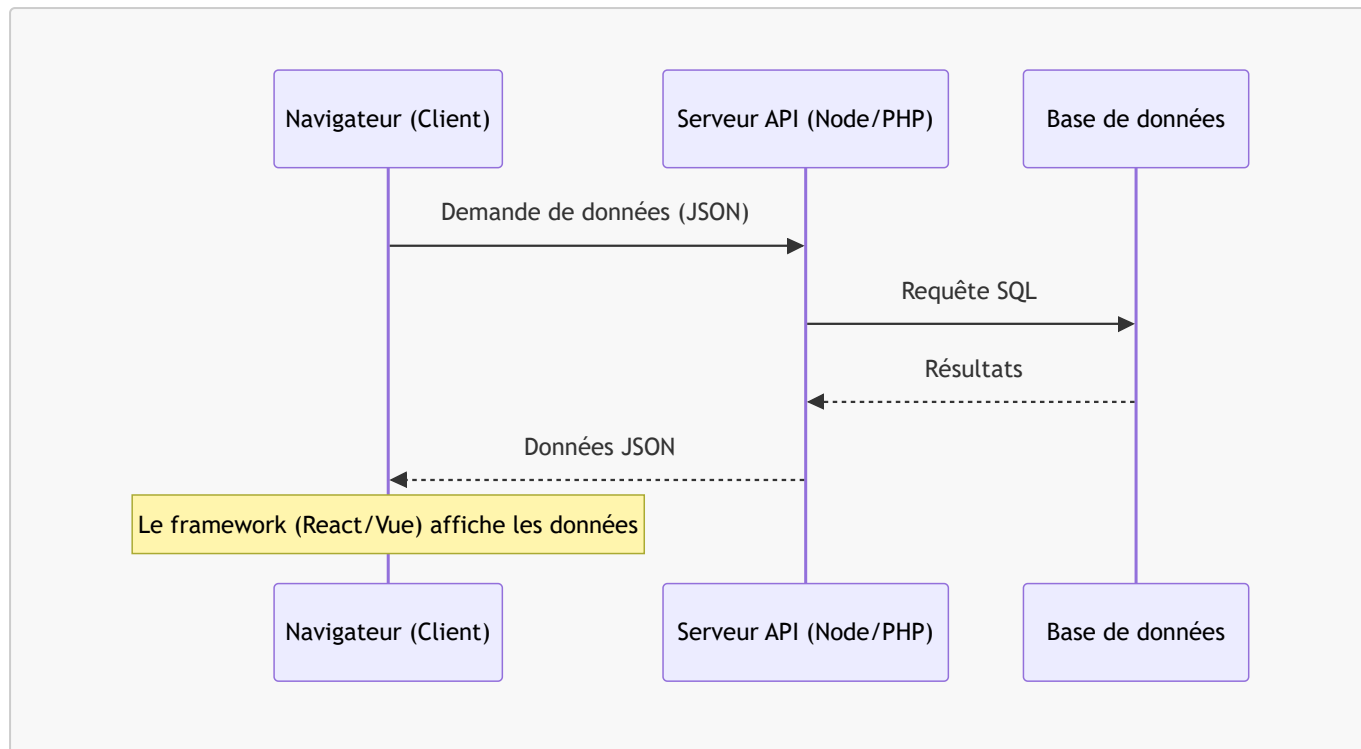
- **Fonctionnement** : Le CMS (ex: WordPress, Drupal) utilise le langage **PHP** pour interroger une base de données **SQL** et générer du **HTML** à la volée.
- **Composants clés** :
 - **Thèmes** : La couche de présentation (CSS).
 - **Plugins / Widgets** : Extensions de fonctionnalités (formulaires, e-commerce).
 -

Dans l'écosystème actuel, les **CMS** (*Content Management Systems*) se divisent en deux grandes familles selon leur philosophie d'affichage. Les solutions classiques **avec interface**, comme **WordPress**, **Joomla** ou **Drupal**, sont dites « couplées » : elles fournissent à la fois l'outil de gestion de contenu et le moteur de rendu visuel (le thème). À l'inverse, les **CMS Headless** (sans interface frontale), tels que **Strapi**, **Contentful** ou **Directus**, fonctionnent comme de purs réservoirs de données. Ils ne dictent pas l'apparence du site mais mettent le contenu à disposition via une **API** (souvent en format **JSON**), laissant ainsi aux développeurs une liberté totale pour afficher ces informations sur n'importe quel support (site web en React, application mobile, ou même objet connecté) tout en garantissant une sécurité accrue et de meilleures performances.

2.2 Frameworks Front-End : La révolution React/Vue

Aujourd'hui, le serveur envoie souvent une application "vide" et c'est le navigateur qui construit la page grâce à des frameworks JavaScript.

- **SPA (Single Page Application)** : Le site ne se recharge jamais. Seules les données (format **JSON**) transitent entre le serveur et le client.
- **API (Application Programming Interface)** : Le serveur web devient une simple source de données (souvent avec **Node.js** ou **PHP 8**).



2.3 Thèmes et plugins

Dans la version actuelle de WordPress (basée sur l'architecture logicielle de 2026), la gestion des pages et de l'apparence a totalement convergé vers une approche visuelle unifiée. Voici une analyse détaillée de ces mécanismes :

La Gestion des Pages et des Contenus : L'Ère du Full Site Editing

La gestion des contenus ne se limite plus à la simple saisie de texte. Avec l'évolution de l'**Éditeur de blocs (Gutenberg)**, chaque page est désormais traitée comme un assemblage de composants modulaires. Cette approche permet de structurer des mises en page complexes (colonnes, bannières, grilles de produits) sans toucher à une seule ligne de code. Les contenus sont enregistrés de manière structurée dans la base de données **MySQL**, tandis que l'interface permet une prévisualisation en temps réel (WYSIWYG - *What You See Is What You Get*). La gestion des révisions et des flux de publication assure une traçabilité totale, essentielle pour la sécurité et l'intégrité des données.

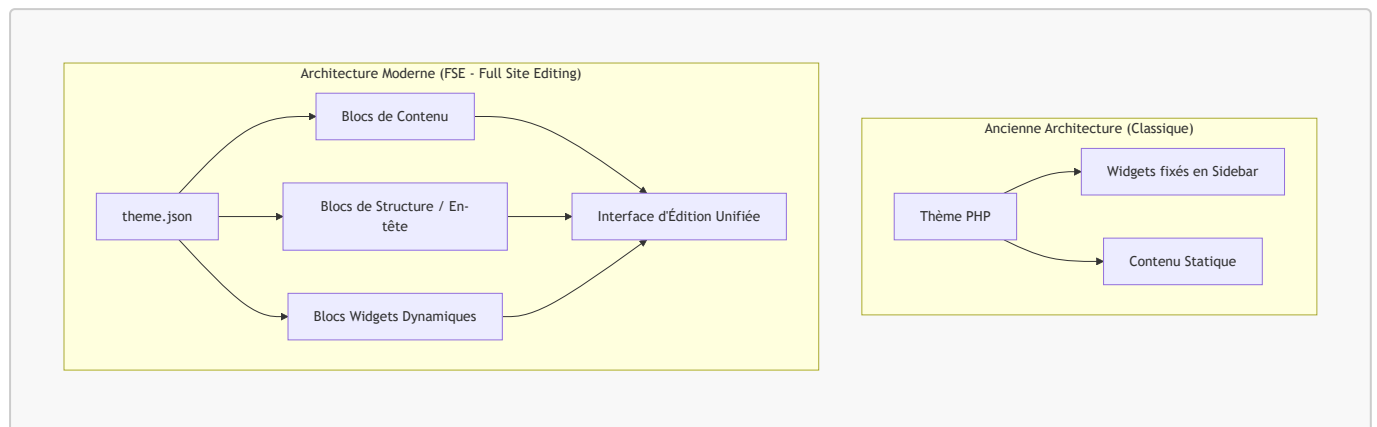
Thèmes et Compositions (Patterns) : La Flexibilité Totale

Les thèmes modernes de WordPress sont désormais des **Thèmes de Blocs**. Contrairement aux thèmes classiques qui utilisaient des fichiers PHP rigides pour chaque section (en-tête, pied de page), les thèmes actuels reposent sur des fichiers HTML et des fichiers de configuration **JSON** (`theme.json`). Cette structure permet une personnalisation granulaire des styles globaux (couleurs, typographies, espacements) directement depuis l'interface d'administration. Les "Compositions" (*Patterns*) ont pris une place centrale : ce sont des ensembles de blocs pré-configurés que l'on peut insérer en un clic pour créer des sections entières de site, garantissant une cohérence esthétique professionnelle.

L'Évolution des Widgets : Vers une Disparition au Profit des Blocs

Le concept historique des **Widgets** — ces petits blocs fonctionnels que l'on plaçait dans les barres latérales ou les pieds de page — a quasiment fusionné avec celui des blocs. Auparavant limités à des zones spécifiques

définies par le code du thème, les widgets sont désormais des blocs que l'on peut placer n'importe où dans la structure du site. Cette mutation offre une liberté totale : vous pouvez insérer un widget de "Derniers Articles" ou de "Panier d'Achat" au milieu d'une page de vente, brisant les limites des anciennes architectures web.



Signification et Impact Technique

- **FSE (Full Site Editing)** : Capacité d'éditer l'intégralité du site (y compris les headers et footers) via l'éditeur de blocs.
- **JSON (JavaScript Object Notation)** : Format de fichier utilisé ici pour définir tous les paramètres de style du site de manière légère et performante.
- **Scalabilité** : Cette nouvelle gestion des contenus permet au site de rester performant même avec un volume de données massif, car le serveur web génère moins de requêtes PHP complexes par rapport aux anciens systèmes.

III. Sécurisation et Mise en Ligne

3.1 HTTPS et TLS : Chiffrer le tunnel

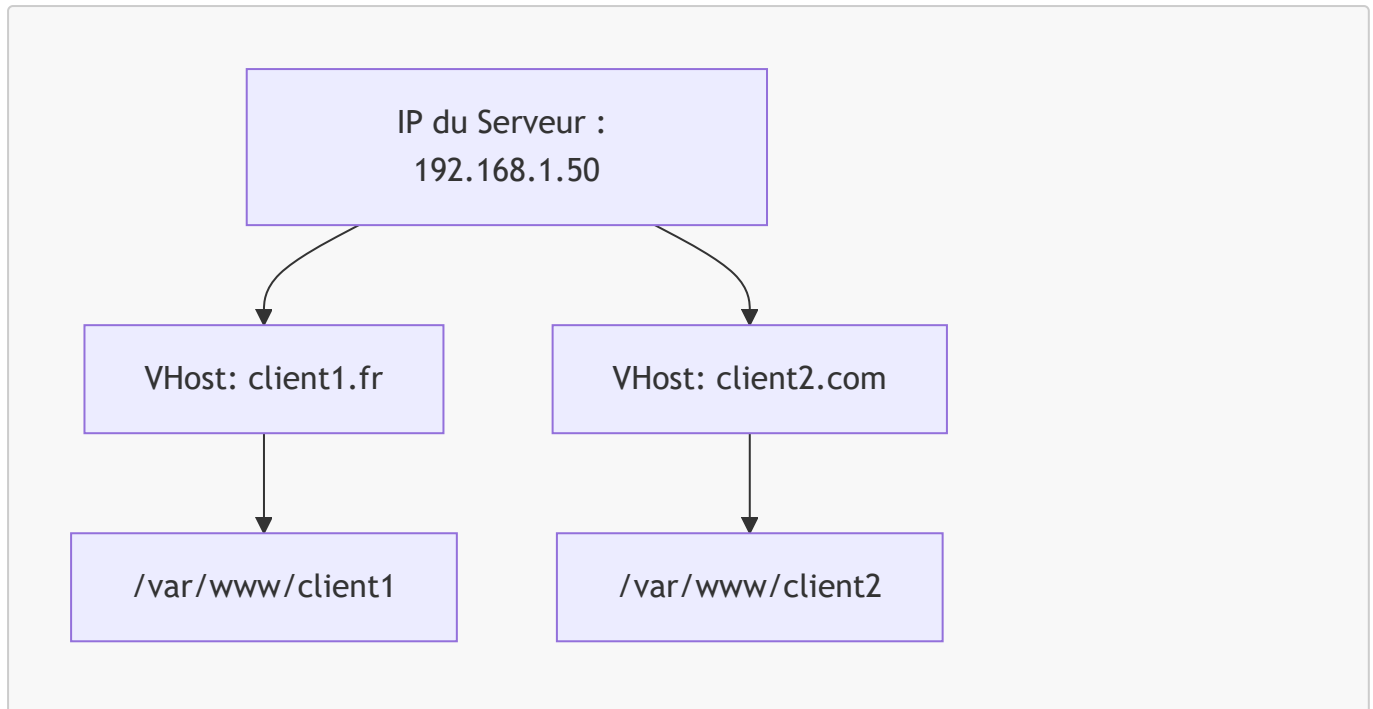
Le passage au **HTTPS** est la priorité numéro 1 de l'administrateur.

- **SSL/TLS (Transport Layer Security)** : Protocole qui chiffre les données entre le navigateur et le serveur.
- **Certificat** : Une carte d'identité numérique délivrée par une Autorité de Certification (ex: *Let's Encrypt*).
- **HSTS (HTTP Strict Transport Security)** : Une instruction envoyée par le serveur qui force le navigateur à ne communiquer **qu'en HTTPS**.

3.2 La Gestion Multi-sites : Hôtes Virtuels (VHosts)

Un serveur possède souvent une seule adresse IP mais doit héberger plusieurs noms de domaine.

- **VHost** : Configuration qui indique au serveur : "*Si la requête demande **site-A.com**, va chercher les fichiers dans le dossier **/var/www/siteA***".



3.4 Mise en ligne d'un site/ hébergement

La mise en ligne est l'étape critique où votre projet quitte votre environnement local pour devenir accessible au monde entier. Voici une analyse détaillée des procédures techniques nécessaires pour réussir ce déploiement.

Choisir un hébergement : Performance et Typologie

Le choix de l'hébergeur détermine la rapidité, la sécurité et la disponibilité de votre site. On distingue trois grandes catégories :

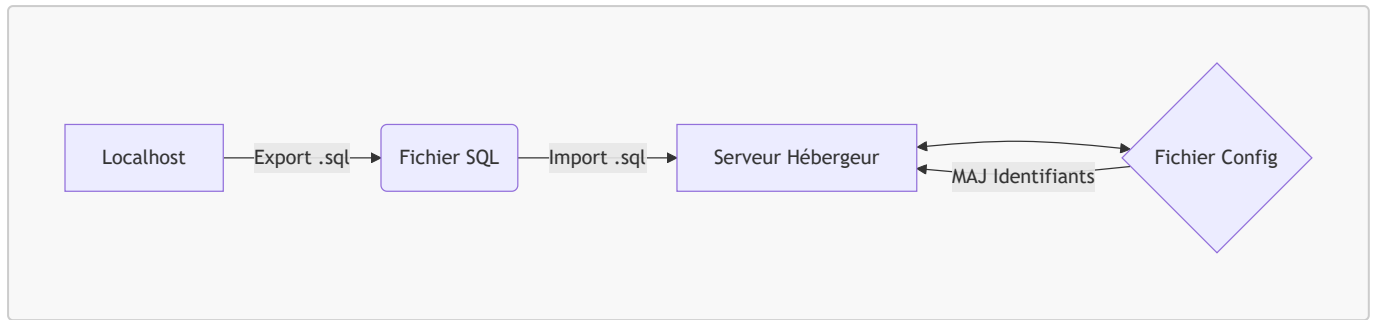
- **L'hébergement mutualisé** : Plusieurs sites partagent les ressources d'un même serveur physique. C'est la solution la plus économique, idéale pour les sites vitrines ou les petits CMS, mais limitée en cas de fort trafic.
- **Le VPS (Virtual Private Server)** : Un serveur physique est découpé virtuellement. Vous disposez de vos propres ressources (RAM, CPU) et d'un accès administrateur total, offrant un excellent compromis entre coût et contrôle.
- **Le Cloud (AWS, Google Cloud, Azure)** : Les ressources sont réparties sur plusieurs serveurs. L'avantage majeur est la **scalabilité** : le serveur s'adapte automatiquement aux pics de charge.

Export et Import de la base de données : Le transfert de l'intelligence

Pour un site dynamique (WordPress, PrestaShop), les fichiers ne suffisent pas ; il faut déplacer la **Base de Données (BDD)** qui contient tous vos textes et réglages.

1. **Export (Dump)** : En local, on utilise un outil comme **phpMyAdmin** pour générer un fichier **.sql**. Ce fichier contient toutes les instructions pour recréer vos tables.
 2. **Import** : Sur le serveur de l'hébergeur, après avoir créé une nouvelle base de données vide, on importe ce fichier **.sql**.
- **Point de vigilance** : Après l'import, il est impératif de modifier le fichier de configuration du site (ex: **wp-config.php**) pour y inscrire les nouveaux identifiants de la base de données fournis par

l'hébergeur.



Copie du site via FTP : Le transfert des fichiers physiques

Une fois la base de données en place, vous devez transférer les fichiers sources (images, feuilles de style CSS, scripts PHP).

- **FTP (File Transfer Protocol)** : C'est le protocole standard pour envoyer des fichiers vers un serveur. On utilise un logiciel client comme **FileZilla** ou **WinSCP**.
- **SFTP (Secure FTP)** : Version sécurisée qui chiffre la connexion via **SSH**. Il est fortement recommandé d'utiliser le SFTP pour éviter que vos identifiants ne circulent en clair sur le réseau.
- **Processus** : On connecte le client FTP grâce à l'adresse du serveur, un identifiant et un mot de passe. Il suffit ensuite de faire glisser les dossiers du répertoire local vers le répertoire distant (généralement nommé `public_html` ou `www`).

Synthèse des abréviations

- **RAM (Random Access Memory)** : Mémoire vive utilisée par le serveur pour traiter les requêtes rapidement.
- **SSH (Secure Shell)** : Protocole permettant d'accéder à un ordinateur distant de manière sécurisée.
- **TTL (Time To Live)** : Durée pendant laquelle une information DNS est mise en cache. Important lors d'un changement d'hébergeur pour que la redirection soit rapide.

IV. Résilience et Maintenance

4.1 Stratégie de Sauvegarde (Backup)

Une sauvegarde professionnelle n'est pas une simple copie de fichiers. Elle doit respecter la règle du **3-2-1** :

- **3** copies des données.
- **2** supports différents (Disque dur, Cloud).
- **1** copie hors-site (géographiquement isolée).

Le duo indispensable :

1. **Fichiers** : Copiés via **SFTP** (Secure FTP) pour conserver les images et scripts.
2. **Base de données** : Exportation au format `.sql` (Dump) pour conserver les textes et utilisateurs.

4.2 Migration et Changement d'Hébergeur

Transférer un site demande une méthodologie stricte pour éviter le "temps mort" (*Downtime*).

1. **Réduction du TTL (Time To Live)** : On prévient les serveurs DNS que l'adresse va changer bientôt.
2. **Migration des fichiers et BDD.**
3. **Test via l'adresse IP** du nouveau serveur.
4. **Mise à jour du DNS.**

4.3 Hébergeurs

Pour choisir un hébergeur, il faut distinguer les acteurs européens (soumis au **RGPD**) des géants américains. Voici les principaux noms du marché en 2026, classés par catégorie :

FR Les Leaders Européens (Souveraineté et Proximité)

- **OVHcloud** : Le numéro 1 français et européen. Offre une gamme complète allant du petit hébergement mutualisé aux serveurs dédiés de haute performance. Idéal pour la conformité des données.
- **o2switch** : Un hébergeur français très populaire pour son "Offre Unique" tout illimité. Excellent rapport qualité/prix pour les CMS (WordPress).
- **Infomaniak** : Basé en Suisse, il se distingue par son approche écologique ("Green IT") et ses services haut de gamme en matière de sécurité et de drive collaboratif.
- **Ionos (by 1&1)** : Géant allemand proposant des solutions très accessibles pour les débutants avec une assistance personnalisée.

Les Géants du Cloud (Scalabilité Mondiale)

- **AWS (Amazon Web Services)** : Le leader mondial. Offre une infrastructure infinie, mais demande des compétences pointues en administration système.
- **Google Cloud Platform (GCP)** : Très performant pour les applications utilisant l'intelligence artificielle et l'analyse de données massive.
- **Microsoft Azure** : La solution de choix pour les entreprises déjà intégrées dans l'écosystème Windows et les serveurs **IIS**.

Les Hébergeurs Spécialisés & Modernes

- **Hostinger** : Connu pour ses prix agressifs et son interface simplifiée (hPanel), très efficace pour les projets internationaux.
- **Vercel / Netlify** : Les leaders pour l'hébergement de sites modernes en **React**, **Vue.js** ou **Next.js**. Ils automatisent le déploiement directement depuis GitHub.
- **SiteGround** : Particulièrement recommandé pour les sites **WordPress** et **WooCommerce** à fort trafic grâce à une optimisation serveur spécifique.

Tableau Comparatif Rapide

Hébergeur	Type de prédilection	Point fort
o2switch	CMS / WordPress	Support technique et prix fixe
OVHcloud	VPS / Dédié	Souveraineté des données (France)
Vercel	Frameworks Front-End	Déploiement continu (Git)

Hébergeur	Type de prédilection	Point fort
AWS	Applications Cloud	Puissance et flexibilité totale

Rappel Sécurité : Quel que soit l'hébergeur choisi, assurez-vous qu'il propose un certificat **SSL** gratuit (Let's Encrypt) et des options de **sauvegarde automatique** quotidiennes.

Glossaire des Abréviations

- **API** : Interface permettant aux applications de s'échanger des données.
 - **DNS** : Système traduisant les noms de domaine en adresses IP.
 - **FTP / SFTP** : Protocole de transfert de fichiers (version sécurisée par SSH).
 - **JSON** : Format de données léger et lisible, standard des échanges Web modernes.
 - **SQL** : Langage de requête pour les bases de données relationnelles.
-